UNIVERSIDAD TECNICA DE MANABI

CARRERA:

TECNOLOGIA DE LA INFORMACION

ACTIVIDAD #1

Estudio de Caso

Sistema de Control de Rescate y Adopción Animal

DOCENTE: Ing. Jorge Zambrano

MATERIA: Desarrollo de Sistemas Informáticos

SEMESTRE: Quinto Semestre

ESTUDIANTE: Suárez Choéz Juan Leonardo

PERIODO ACADEMICO:

septiembre - enero

2026 -2027

Enlace GitHub: https://github.com/JuanLSusrezCh/DES-SIST-INFOR.git

1. INTRODUCCIÓN Y ANÁLISIS DEL PROBLEMA

En los centros de refugio animal, la falta de un sistema centralizado para el control de ingresos, seguimiento veterinario y procesos de adopción genera inconsistencias operativas y pérdida de trazabilidad en los casos. Según Sommerville (2011), el diseño de la arquitectura de software constituye la fase crítica en la que se definen los componentes principales del sistema y sus relaciones, garantizando que la solución sea mantenible y escalable a lo largo del tiempo.

El propósito de esta propuesta es diseñar la arquitectura lógica de una plataforma que coordine las interacciones entre reportantes, personal operativo y administradores, utilizando el estándar de modelado UML (Object Management Group [OMG], 2017) y patrones de diseño orientados a objetos (Gamma et al., 1994).

RESUMEN

Este documento aborda el diseño de la arquitectura lógica para un sistema informático orientado a la gestión y control de rescates y adopciones de animales en un refugio. Se realizo el análisis de requerimientos funcionales y no funcionales, la representación mediante diagramas UML (Casos de Uso, Clases y Secuencia), la aplicación justificada de cuatro patrones de diseño (Singleton, Factory Method, Observer y MVC), y la estructuración del flujo de trabajo colaborativo Gitflow en GitHub.

Actores del Sistema

- **Ciudadano / Reportante:** Persona que notifica el hallazgo de un animal en riesgo y realiza el seguimiento del reporte.

- **Voluntario / Veterinario:** Personal encargado de registrar la atención médica, actualizar estados clínicos y dar seguimiento operativo.

- **Administrador del Refugio:** Gestiona asignaciones de personal, aprueba solicitudes de adopción y administra los usuarios del sistema.

- **Solicitante de Adopción:** Ciudadano que aplica a un proceso de adopción de un animal disponible.

Requerimientos del Sistema

Requerimientos Funcionales (RF):

- **RF-01:** Registrar el caso de rescate con fecha, ubicación, descripción y datos opcionales del reportante.

- **RF-02:** Categorizar el expediente del animal según su especie (Perro, Gato, Fauna Silvestre).

- **RF-03:** Asignar un voluntario o veterinario responsable al caso.

- **RF-04:** Controlar los cambios de estado del caso (*Rescatado*, *En Tratamiento*, *Disponible para Adopción*, *Adoptado*).

- **RF-05:** Enviar notificaciones automáticas al personal asignado ante nuevos eventos.

- **RF-06:** Gestionar el ciclo de solicitudes de adopción.

Requerimientos No Funcionales (RNF):

- **RNF-01 (Mantenibilidad):** Aplicación del patrón desacoplado Model-View-Controller (MVC) y principios SOLID (Martin, 2017).

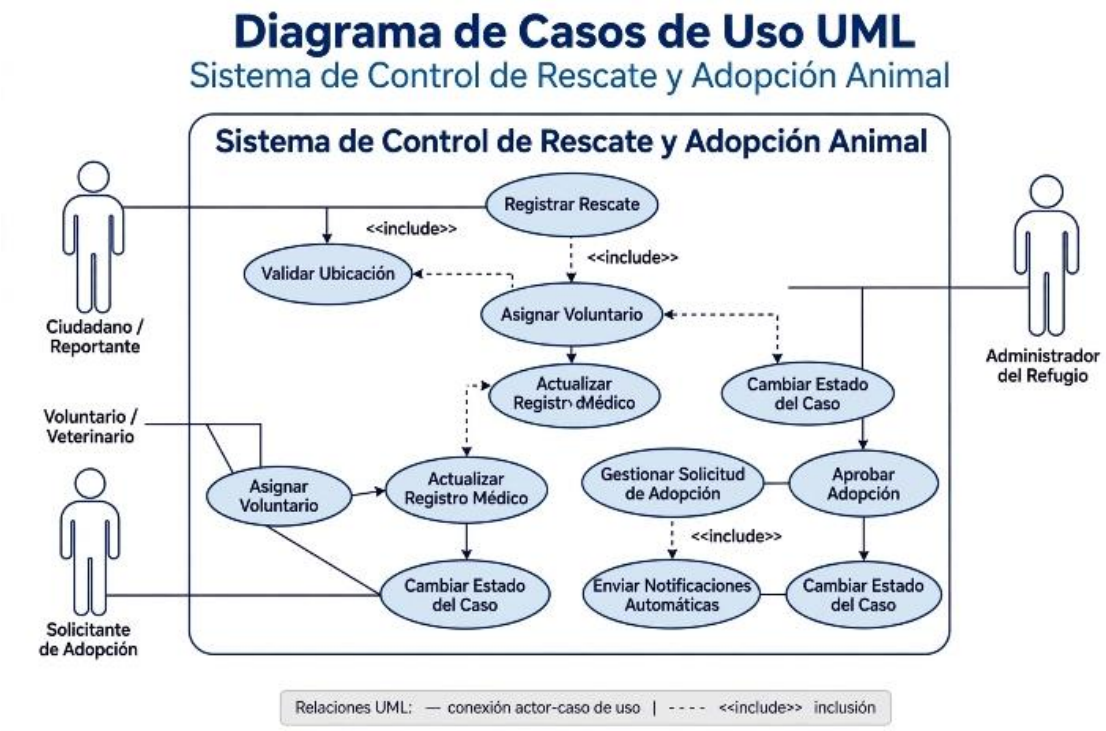
- **RNF-02 (Escalabilidad):** Capacidad para integrar nuevos tipos de animales o canales de notificación sin modificar el núcleo del sistema (Gamma et al., 1994).

- **RNF-03 (Trazabilidad):** Control de versiones estructurado mediante Gitflow (Driessen, 2010).

2. MODELADO DE LA ARQUITECTURA EN UML

El Lenguaje Unificado de Modelado (UML) proporciona la notación estándar para especificar, visualizar y documentar los artefactos de software (OMG, 2017).

2.1 Diagrama de Casos de Uso



- **Relaciones principales:**

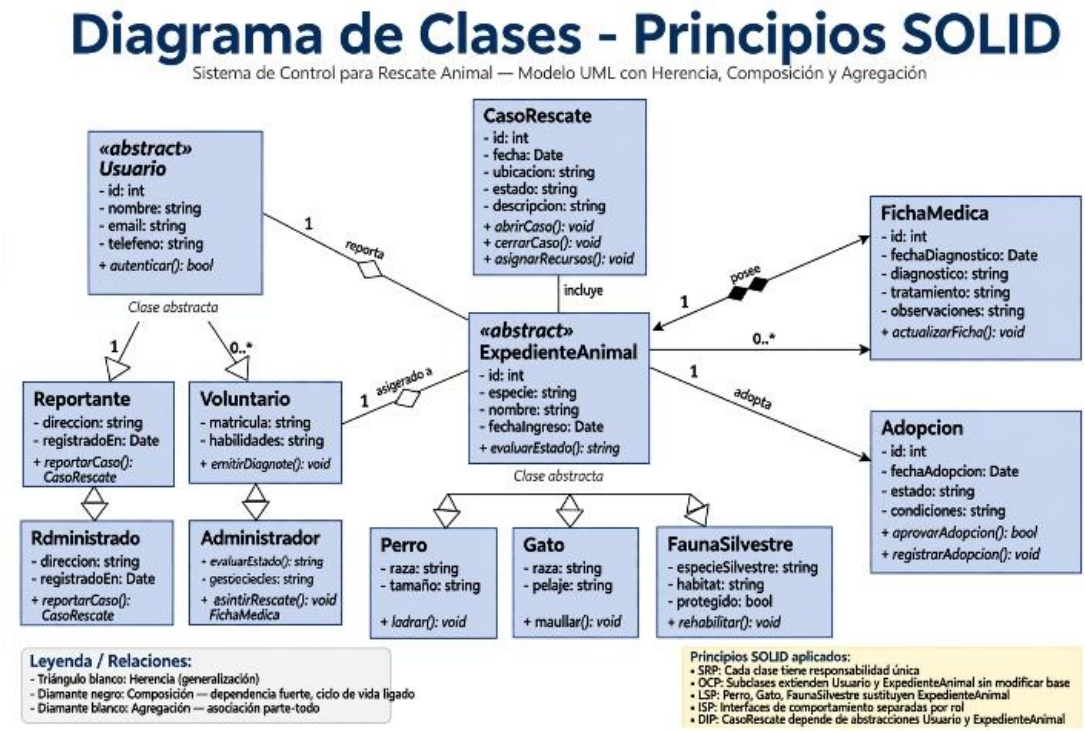
- Ciudadano $\rightarrow$ *Registrar Rescate* (incluye *Validar Ubicación*).
- Voluntario/Veterinario $\rightarrow$ *Actualizar Ficha Médica* y *Cambiar Estado de Caso*.
- Administrador $\rightarrow$ *Asignar Encargado* y *Aprobar Adopción*.

2.2 Diagrama de Clases UML y Principios SOLID

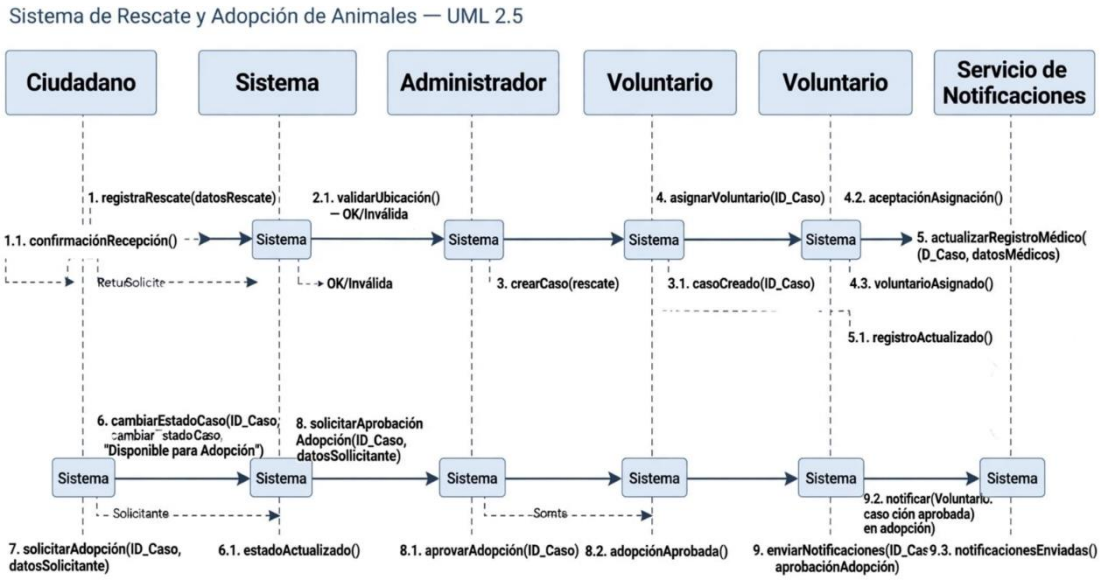
El diseño orientado a objetos aplica los principios SOLID expuestos por Martin (2017) para garantizar la cohesión y reducir el acoplamiento:

- **Single Responsibility Principle (SRP):** La clase CasoRescate administra el ciclo de vida del proceso, mientras que la clase ServicioNotificacion se encarga exclusivamente de las alertas.
- **Open/Closed Principle (OCP):** La clase abstracta Animal permite derivar clases especializadas (Perro, Gato, FaunaSilvestre) sin alterar las clases controladoras.

- **Dependency Inversion Principle (DIP):** Los controladores interactúan mediante la interfaz INotificador, desacoplándose de implementaciones específicas (Email, SMS, Push).



2,3 Diagrama de Secuencias – Flujo de Rescate y Adopción



3. PATRONES DE DISEÑO APLICADOS Y JUSTIFICACIÓN TÉCNICA

Se seleccionaron e implementaron cuatro patrones de diseño fundamentales definidos por Gamma et al. (1994) y estructurados arquitectónicamente por Microsoft (2020):

Patrón	Clasificación	Función en el Sistema de Rescate	Justificación Técnica
Singleton	Creacional	Gestión del pool de conexiones a la base de datos y del motor central de casos.	Garantiza una única instancia global (ConexionBD), optimizando el consumo de recursos de memoria y conexiones concurrentes (Gamma et al., 1994).
Factory Method	Creacional	Instanciación heterogénea de expedientes según la especie (Perro, Gato, FaunaSilvestre).	Abstrae la lógica de creación en ExpedienteFactory, permitiendo instanciar objetos con atributos médicos específicos según el tipo sin acoplar el código cliente (Gamma et al., 1994).
Observer	Comportamiento	Sistema de alertas reactivas ante el registro o cambio de estado de un rescate.	El sujeto (CasoRescate) notifica automáticamente a los observadores suscritos (VoluntarioObserver, VeterinarioObserver) al cambiar de estado, aplicando programación reactiva (Gamma et al., 1994).
MVC	Arquitectónico / Estructural	Separación de capas: Modelo	Desacopla la lógica del negocio de la

Patrón	Clasificación	Función en el Sistema de Rescate	Justificación Técnica
		(CasoRescate), Vista (UIWeb), Controlador (RescateController).	interfaz de usuario y la persistencia de datos, mejorando la mantenibilidad y testabilidad del código (Microsoft, 2020).

4. GESTIÓN DEL DISEÑO CON GITFLOW Y GITHUB

Para el registro del avance del proyecto se adoptó la estrategia de ramificación Gitflow descrita por Driessen (2010), la cual organiza el desarrollo colaborativo en ramas con responsabilidades explícitas.

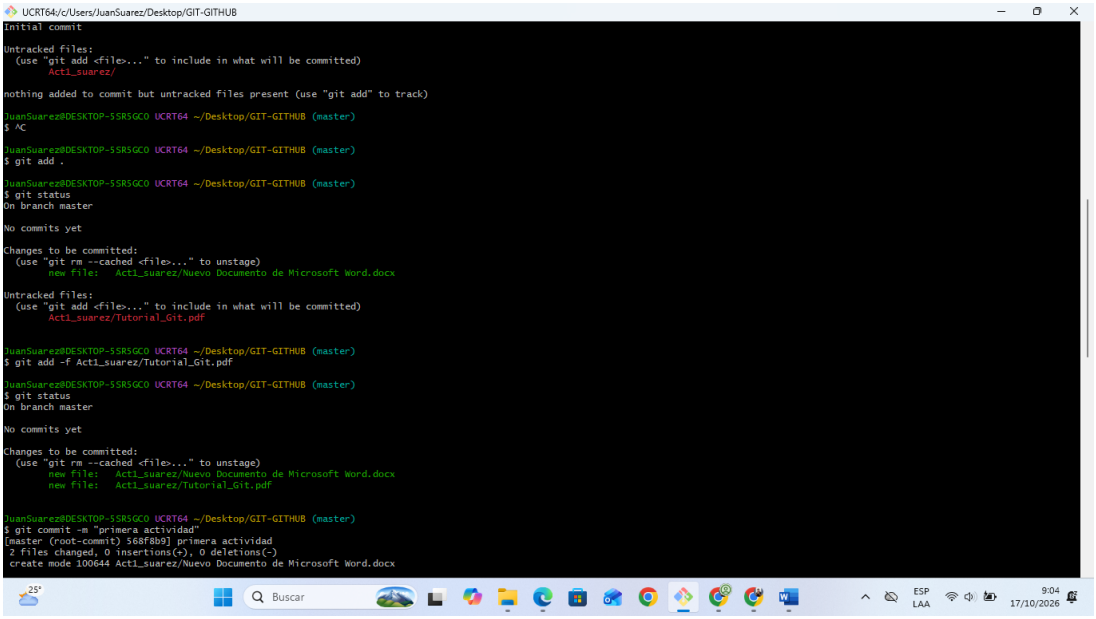
4.1 Estrategia de Ramas

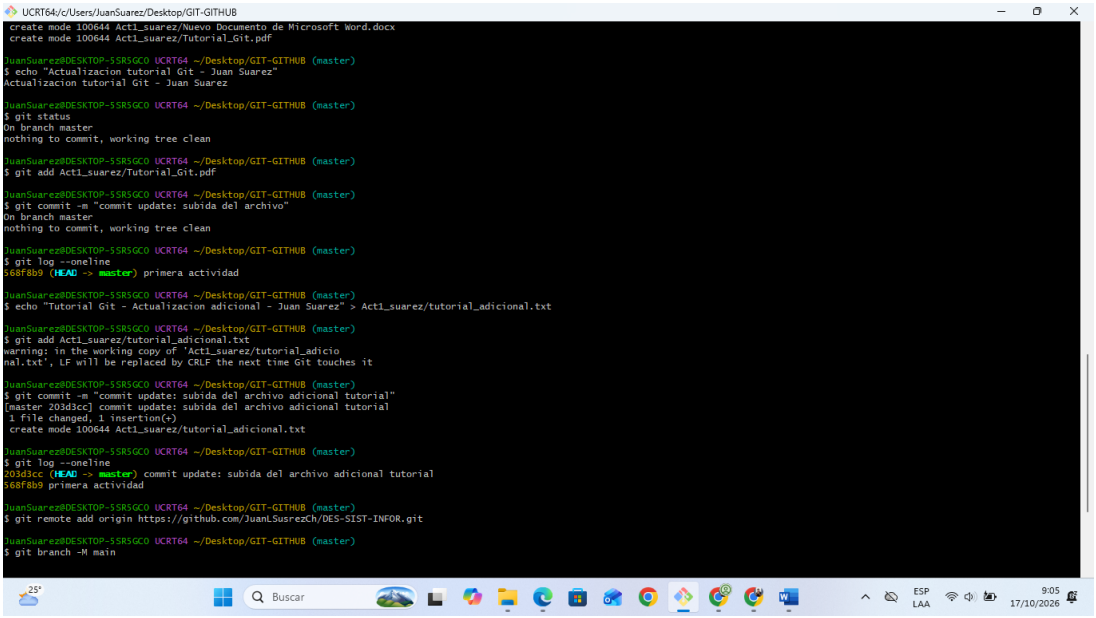
- main / master: Contiene las versiones estables y entregas oficiales.
- develop: Rama base de integración continua.
- feature/casos-uso: Trabajo en el modelado de casos de uso.
- feature/diagrama-clases: Diseño UML de clases y principios SOLID.
- feature/patrones-diseno: Modelado e implementación de Singleton, Factory y Observer.

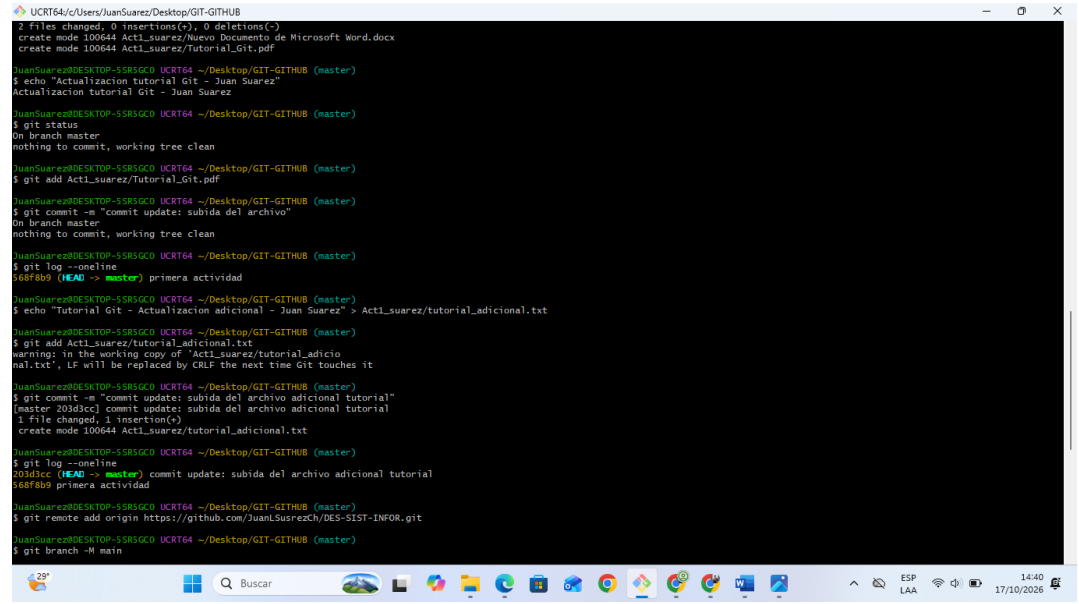
4.2 Repositorio y Registro de Commits

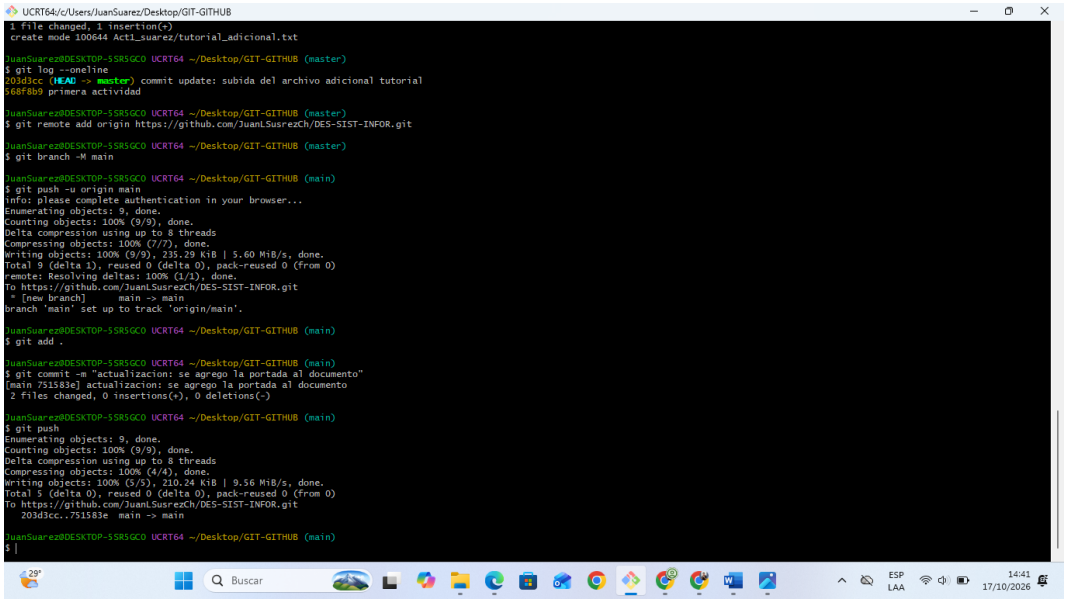
Enlace al repositorio público:
https://github.com/JuanLSusrezCh/DES-SIST-INFOR.git

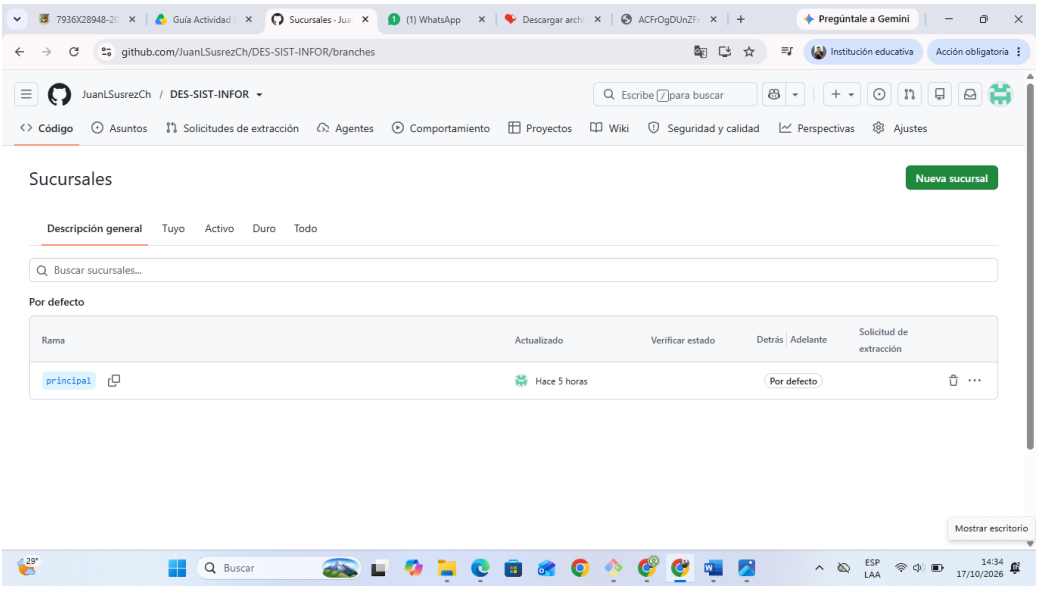
Secuencia de comandos ejecutados:

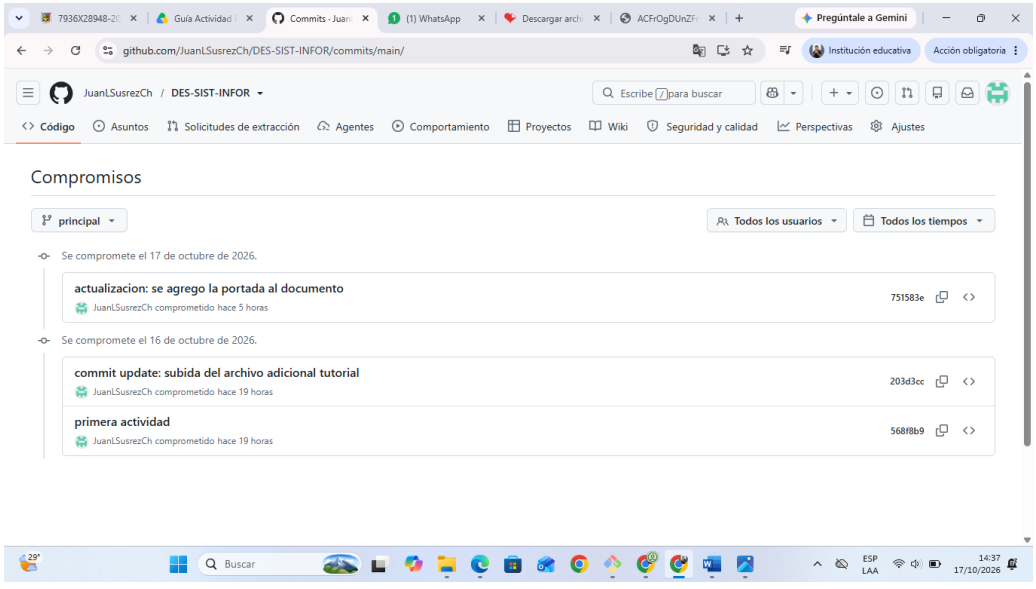







UCRT64:/c/Users/JuanSuarez/Desktop/GIT-GITHUB
1 file changed, 1 insertion(+)
create mode 100644 Act1_suarez/tutorial_adicional.txt
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (master)
$ git log --oneline
203d3cc (HEAD -> master) commit update: subida del archivo adicional tutorial
568f8b9 primera actividad
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (master)
$ git remote add origin https://github.com/JuanLSusrezCh/DES-SIST-INFOR.git
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (master)
$ git branch -M main
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 235.29 KiB | 5.60 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/JuanLSusrezCh/DES-SIST-INFOR.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (main)
$ git add .
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (main)
$ git commit -m "actualizacion: se agrego la portada al documento"
[main 751583e] actualizacion: se agrego la portada al documento
2 files changed, 0 insertions(+), 0 deletions(-)
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (main)
$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 210.24 KiB | 9.56 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/JuanLSusrezCh/DES-SIST-INFOR.git
203d3cc..751583e main -> main
JuanSuarez@DESKTOP-5SR5GCO UCRT64 ~/Desktop/GIT-GITHUB (main)
$


github.com/JuanLSusrezCh/DES-SIST-INFOR/branches
JuanLSusrezCh / DES-SIST-INFOR
Código
Asuntos
Solicitudes de extracción
Agentes
Comportamiento
Proyectos
Wiki
Seguridad y calidad
Perspectivas
Ajustes
Sucursales
Nueva sucursal
Descripción general
Tuyo
Activo
Duro
Todo
Buscar sucursales...
Por defecto
Rama
Actualizado
Verificar estado
Detrás
Adelante
Solicitud de extracción
principal
Hace 5 horas
Por defecto
Mostrar escritorio


github.com/JuanLSusrezCh/DES-SIST-INFOR/commits/main/
JuanLSusrezCh / DES-SIST-INFOR
Código
Asuntos
Solicitudes de extracción
Agentes
Comportamiento
Proyectos
Wiki
Seguridad y calidad
Perspectivas
Ajustes
Compromisos
principal
Todos los usuarios
Todos los tiempos
Se compromete el 17 de octubre de 2026.
actualizacion: se agrego la portada al documento
JuanLSusrezCh comprometido hace 5 horas
751583e
Se compromete el 16 de octubre de 2026.
commit update: subida del archivo adicional tutorial
JuanLSusrezCh comprometido hace 19 horas
203d3cc
primera actividad
JuanLSusrezCh comprometido hace 19 horas
568f8b9

5. CONCLUSIONES

La aplicación integrada de los patrones **Factory Method** y **Observer** resolvió la flexibilidad requerida para categorizar especies y notificar eventos en tiempo real sin acoplar los componentes del sistema (Gamma et al., 1994).

La arquitectura basada en el patrón **MVC** y los principios **SOLID** asegura la mantenibilidad del código ante futuros requerimientos del refugio animal (Martin, 2017; Microsoft, 2020).

El flujo de trabajo **Gitflow** proporcionó un orden estricto en la trazabilidad del diseño dentro de GitHub (Driessen, 2010).

REFERENCIAS BIBLIOGRÁFICAS

1. **Driessen, V.** (2010). *A successful Git branching model*. nvie.com. https://nvie.com/posts/a-successful-git-branching-model/

2. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

3. **Martin, R. C.** (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

4. **Microsoft.** (2020). *ASP.NET Core MVC Overview*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/mvc/overview

5. **Object Management Group [OMG].** (2017). *Unified Modeling Language (UML) Specification Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

6. **Sommerville, I.** (2011). *Software Engineering* (9.ª ed.). Addison-Wesley.